

1.5.4 Multitasking or Time Sharing System

Multiprogramming provides an environment in which various system resources (viz. CPU, memory, I/O devices) were utilized effectively, but it did not provide user interaction with the computer system. Time sharing (or Multitasking) is a **logical extension** of Multiprogramming.

In such systems there are more than one user interacting the system at the same time. CPU bound is divided into different time slots depending upon the number of users using the system.

The switching of CPU between two user is so fast that it gives the impression to user that he is only working on the system but actually it is shared among different users.

A Multitasking system uses **CPU scheduling** and multiprogramming to provide each user with a small portion of a time-shared computer. Each user has at least one separate program in memory.

Multitasking operating system are more complex then Multiprogramming operating system. In both, several jobs are kept simultaneously in memory; for this some **memory management schemes** are required.

Multitasking system must provide a mechanism for jobs **synchronization** and **communication** and it may ensure that system does not go in **deadlock state**.

Although, batch processing is still in use but most of the systems today available uses the concept of Multitasking and Multiprogramming. So, we can say that Multitasking (or Time sharing) and Multiprogramming are the central themes of modern OS and they are central themes of this book.

1.5.5 Network Operating System

An operating system, which includes software to communicate with other computers via a network, is called network operating system. This allows resources such as files, application programs, and printers to be shared among computers. Such operating system are specialized to provide various networking services.

For example, BSD (Berkeley System Distribution) Unix, Novell, Lantastic, MS LAN Manager, Windows NT etc.

Network Operating System serves as a peer-to-peer operating system or as a client-server operating system.

1.5.5.1 Client-Server Operating System

A Client server machine (figure 1.8) requires some specific software components. A computer system in which server program is installed to provide various services for their client is called server while other machines on which depends on server machine to provide their services is called as client.

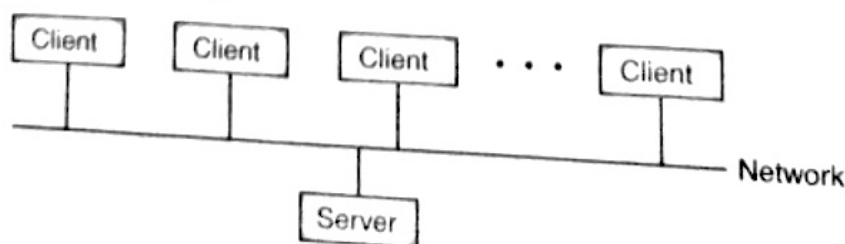


Fig. 1.8.

A machine which works as a server cannot provide any client functionality. On a **Novell server** user cannot run word processing application whereas on **Microsoft's NT server** and **UNIX server** can run client programs also.

1.5.5.2 Peer-to-Peer Operating System

A computer in a Peer-to-peer Network can work as a server or client, i.e. it requires both server and client softwares. **Windows NT** and **Windows 95** both are Peer-to-Peer operating system which include many services and utilities that facilitates networking.

1.5.6 Distributed Operating System

Distributed operating system are the operating system for a network of **autonomous** computers connected by a **communication network** through a message passing mechanism. A distributed operating system controls and manages the hardware and software resources of a distributed system. When a program is executed on a distributed system, user is not aware of where the program is executed or the location of the resources accessed.

The basic design issue of a distributed OS is same as in traditional operating system, viz. process synchronization, deadlocks, scheduling, file systems, inter-process communication, memory management etc.

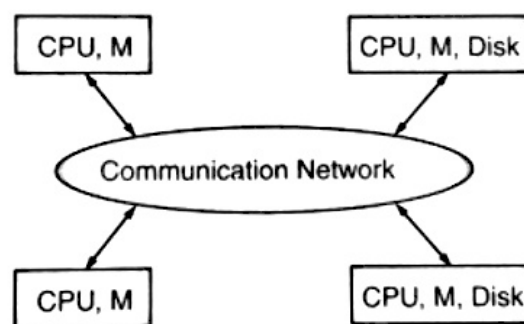


Fig. 1.9 Architecture of Distributed systems

But some additional feature like the **absence** of both shared memory and global clock, and unpredictable communication delays make the design of distributed operating system very difficult. For example: **Alpha, Kernel, Amoeba, Mach** are few examples of distributed operating systems.

4.5.5.1 Peer-to-Peer Operating System

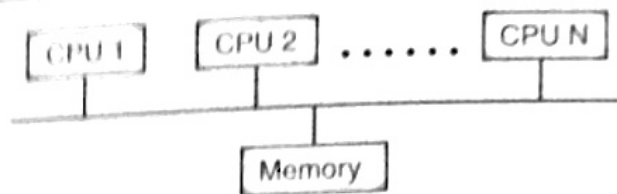


Fig. 1.10 Multiprocessor system

1.5.8 Real Time Operating System

Primary objective of Real Time Operating system is to provide quick response time and thus to meet a scheduling deadline. User convenience and resource utilization are **secondary** concern to these systems.

Real time systems has many events that must be accepted and processed in a short time or within certain deadline. Such applications include:

Rocket launching, flight control, robotics, real time simulation, telephone switching equipment etc.

Real time systems are classified into two categories:

- (i) **Soft Real Time System:** If certain deadlines are missed then system continues its working with no failure but its performance degrades.
It is a **less restrictive** type of real time system, where a critical real time task of gets priority over other tasks, and retains the priority until it completes. Soft Real time systems have **more limited utility** then hard real time systems. These systems require advance operating system features that cannot be supported by hard real-time systems. They are risky to use for industrial control and robotics. They are useful in multimedia, virtual reality and advanced scientific projects.
- (ii) **Hard Real Time System:** If any deadline is missed then system will fail to work or does not work properly.

This system guarantees that critical tasks be completed on time. Virtual memory (ch. 8) is never found in a real time system. So, hard real time systems conflict with the operation of time-sharing systems, and the two cannot be mixed. Since more of the existing general purpose operating system support hard real time functionality, we do not concern ourselves with this type of system in this text.

1.5.9 Modern Operating System

Users engage in diverse activities in a modern computing environment. Hence, a modern operating system cannot use a uniform strategy for all processes. It must use a strategy that is appropriate for each individual process. For example, a user may open a mail handler, edit files, execute some programs and may watch a video at the same time. Here, execution of a program may be interactive, and may involve an activity in another node of a distributed system and video watching is a soft real time activity. So, the operating system must use Round-Robin scheduling for program execution, use priority-based scheduling for processes of the video application, and implement Remote Procedure Calls (RPC) to support activities in another node. Thus, a modern operating system uses most concepts and techniques that we discussed in connection with the batch processing, multiprogramming, time sharing, real time and distributed operating systems.

PROCESS MANAGEMENT

A process is an instance of a program in execution. The concept of a process, which forms the subject matter of current chapter, is central to the study of operating systems. All multiprogramming operating systems from single-user systems to mainframes that can support thousand of users, are built around the concept of the process. A process will need certain resources to accomplish its task. These resources are processor, memory, files and input/output devices.

A process is the smallest unit of work individually schedulable by an operating system. Such a system consists of a collection of processes: operating system processes execute system code, and user processes execute user code. All these processes may execute concurrently. Potential benefits of concurrent execution of programs include increased performance, resource utilization, and responsiveness of a computer system.

Processes are a primary mechanism, for defining and managing concurrent execution of programs under control of an operating system.

The operating system is responsible for the following activities in connection with **process** and **thread management**: the **creation** and **deletion** of both user and system processes; and the provision of mechanisms for **communication**, **synchronization** and **deadlock handling** for processes.

3.1 | THE PROCESS CONCEPT

The concept of a process, which was first used by the designers of MULTICS, is an essential part of the theory and practice of modern operating systems.

The processor has to manage several activities at one time—each activity corresponds to one process.

The term 'task' is also used instead of 'process'. In some situation 'task' is preferable, since the word process can be usefully employed in a general sense—but not in the discussion of operating systems. Also the word multiprocessing means "multiple processes running at the same time" which raises the problem of what word do we use to describe a computer with multiple processors. However the term 'process' is now widely used and accepted.

The environment in which we interact to use a computer is built out of process. Command interpreter is a process. When we execute a program that we have just compiled,

the operating system generates a process to run the computer. Almost all the activities performed by the processor is a process.

Process is **not** same as program. A program is a **passive entity** while a process is an **active entity**. We can distinguish between a program, such as the contents of files stored on disk and a process, with a program counter specifying the next instruction to execute.

Many people can run the same program; and each copy of this program corresponds to a distinct process.

Although two processes may be associated with the same program, they nevertheless considered two separate execution sequences. For instance, several users may be running different copies of the mail program, OR the same user may use many copies of the editor program. Each of these is a separate process and although the **text** sections are equivalent, the **data** section may vary.

EXAMPLE 3.1 This example illustrates the difference between a process and a program.

```
main ( )
{
    int i, prod = 1;
    for (i = 0; i < 100; i++)
        prod = prod * i;
}
```

It is a program containing one multiplication statement ($\text{prod} = \text{prod} * i$); but the process will execute 100 multiplication, one at a time through the 'for' loop.

3.2 | PROCESS STATES

When a process comes in execution it changes its state. The state of a process is defined in part by the current activity of that process. Each process may be in one of the following state during the time of its execution.

- **New** : In this state, the process is being created.
- **Ready** : In this state, the process is waiting to be assigned to a processor through a short-term scheduler.
- **Waiting** : In this state the process is waiting OR blocked for some event to occur such as an input/output completion on reception of a signal.
- **Running** : In this state, instructions are being executed.
- **Terminated** : In this state, the process has finished the execution.

These state names are arbitrary, and they vary across operating systems. However, the states that they represent are found on all systems.

In a uniprocessor system only one process can be in a running state at a single instant of time, although many processes may be in ready OR waiting state.

The **state transition diagram** corresponding to these states is presented in **Figure 3.1**.

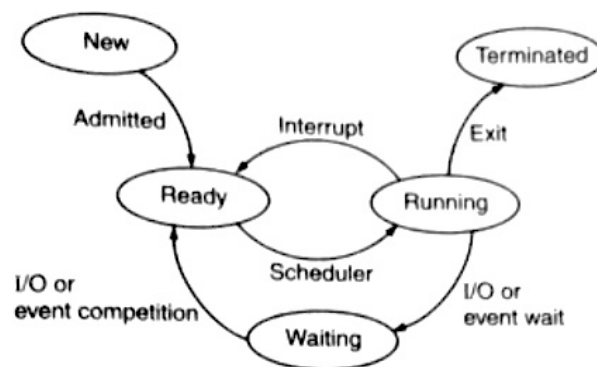


Fig. 3.1 Process State Transition diagram

3.3 | PROCESS CONTROL

3.3.1 Process Control Block (PCB)

In operating system each process is represented by a process control block (PCB) OR a task control block (Figure 3.2).

It is a data structure that physically represent a process in the memory of a computer system. It contains many pieces of information associated with a specific process that includes the following :

Pointer	Process state
Process number	
Program counter	
Registers	
Memory limits	
List of open files	
⋮	

Fig. 3.2 Process control block (PCB)

- **Process state** : The state may be new, ready, waiting, running, terminated etc.
- **Program counter** : The counter indicates the address of the next instruction to be executed for this process.
- **CPU registers** : The registers vary in number, size and type, depending upon the computer architecture. They include accumulators, index registers, flags, stack pointers and other general purpose registers, plus any condition-code information.
- **CPU scheduling information** : This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters (chapter 5 describes process scheduling).

- **Memory-management information** : This information includes the value of base and limit registers, the page or segment tables, depending upon the memory management scheme used by the operating system (chapter 7).
- **Accounting information** : This information includes the amount of CPU and real time used, time limits, account numbers, job on process numbers and so on.
- **Input/output information** : The information includes the list of input/output devices allocated to this process, a list of open files, and so on.

The PCB serves as the repository for any information that may vary from process to process. The content, format and name of this structure will vary, but some characteristics will be similar from system to system.

3.3.2 Interleaving Execution of Processes

From the user point of view, multiple processes are executed concurrently (as shown in figure 3.3) with respect to time.

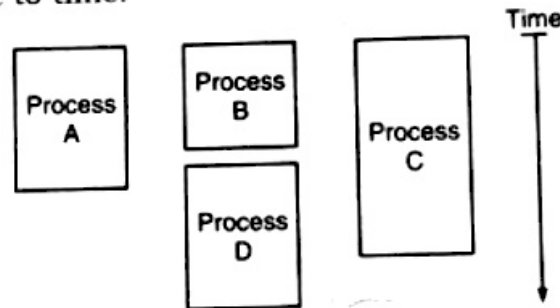


Fig. 3.3 Process execution user point of view.

In reality, operating system interleaves the process execution (as shown in figure 3.4) with respect to time.

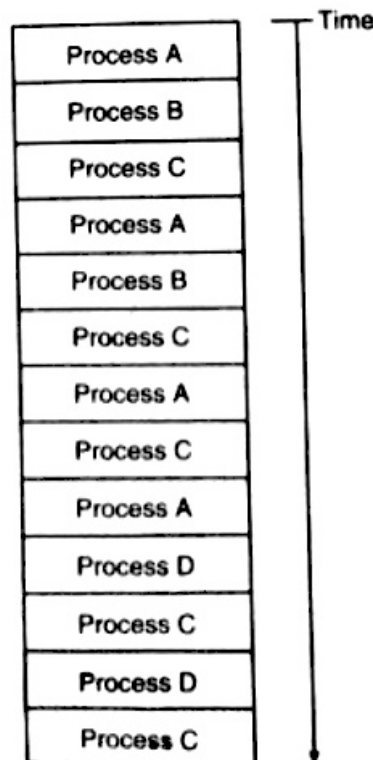


Fig. 3.4 Process execution system point of view

3.3.3 Modes of Execution

A process in an operating system can be executed either in **system mode** (OR kernel mode) OR in **user mode**.

An operating system may prevent the execution of some instructions in user mode but allow them to be executed only in privileged mode (also called system mode or kernel mode). These instructions are :

- Read/write a control register such as PSW
- Primitive input/output and memory management.

The two modes of execution protect the operating system **data structures** from the interference of the user code. System mode (Kernel mode) provides full control of the system that may not be needed for user programs. The system mode can be entered by setting a bit of information in PSW,

The system can enter in privileged mode as a result of a request from user code and return to user mode after completing the request.

3.3.4 Process Switching

Switching from one process to another in a system requires saving the state of the old process and loading the saved state for the new process. This task is known as **context switching**. The context of a process is represented in the PCB of a process that includes various informations such as the value of CPU registers, the process state, memory management information, scheduling information etc. When a context switch occurs, the kernel saves the context of old process in its PCB and loads the context of next (new) process that is currently scheduled to run (Figure 3.5). The switching time between two process is pure overhead, because the system does no useful work, while switching. Its speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instruction to store OR load registers. Typical speed ranges from 1 to 1000 microseconds.

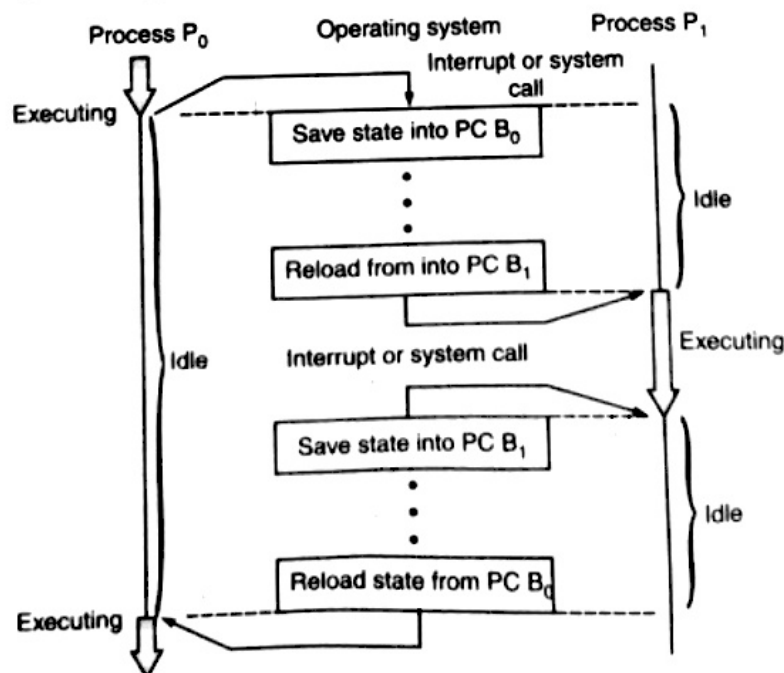


Fig. 3.5. Diagram showing CPU switch from one process to another